



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

**IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED
REASONING (RBR), AND CASE-BASED REASONING (CBR)**

Student Name	Jamella Tracy L. Dancel
Section:	AN42
Professor:	Ms. Crisola G. Tan
Date:	October 5, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge.

Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware

- A laptop or desktop computer
- Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
(Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {  
    "temperature": 32,  
    "is_raining": False,  
    "humidity": 85,  
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):  
    if facts["temperature"] > 30:  
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:

- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [  
    {"problem": {"won't turn on": True, "noise": False},  
     "solution": "Check power cable."}  
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks

```
# =====  
# PART 1: Knowledge Representation (KR) Setup  
# =====  
print("=== PART 1: Knowledge Representation Setup ===")  
print("Let's set up your initial Knowledge Base (KB).")  
  
# Interactive input for KB facts
```

```

try:
    temp_input = int(input("Enter current temperature (e.g., 34): ") or 34)
    humidity_input = int(input("Enter humidity percentage (e.g., 80): ") or 80)
    rain_input = input("Is it raining? (true/false, default: true): ").strip().lower() == 'true'
    motion_input = input("Is motion detected? (true/false, default: true): ").strip().lower() ==
'true'
    door_input = input("Is the door open? (true/false, default: false): ").strip().lower() == 'true'
except ValueError:
    print("Invalid input detected. Using default baseline values.")
    temp_input, humidity_input, rain_input, motion_input, door_input = 34, 80, True, True,
False

knowledge_base = {
    "temperature": temp_input,
    "humidity": humidity_input,
    "is_raining": rain_input,
    "motion_detected": motion_input,
    "door_open": door_input
}

print("\n[Output] Knowledge Base initialized successfully:")
for category, fact in knowledge_base.items():
    print(f" - {category}: {fact}")
print("-" * 50)

# =====
# PART 2: Implement Rule-Based Reasoning (RBR)
# =====
print("=== PART 2: Rule-Based Reasoning (RBR) ===")

# Task 2A: Create at least 3 IF-THEN rules as Python functions
def rule_hot(facts):
    if facts.get("temperature", 0) > 30:
        return "Turn on the aircon"
    return None

def rule_rain(facts):
    if facts.get("is_raining", False):
        return "Bring umbrella and close windows"
    return None

```

```

def rule_security(facts):
    if facts.get("motion_detected", True) and facts.get("door_open", False):
        return "Trigger security alert"
    return None

# List of rules
rules_list = [rule_hot, rule_rain, rule_security]

# Task 2B: Implement an Inference Engine
def inference_engine(kb, rules):
    inferred_actions = []
    for rule in rules:
        action = rule(kb)
        if action:
            inferred_actions.append(action)
    return inferred_actions

# Task 2C: Display Output
print("Rule-Based Reasoning Actions:")
actions = inference_engine(knowledge_base, rules_list)
if actions:
    for action in actions:
        print(f" - {action}")
else:
    print(" - No rules triggered with current facts.")
print("-" * 50)

# =====
# PART 3: Implement Case-Based Reasoning (CBR)
# =====
print("=== PART 3: Case-Based Reasoning (CBR) ===")

# Task 3A: Build a Case Base
case_base = [
    {
        "problem": {"won_t_turn_on": True, "strange_noise": False, "indicator_light_off": True},
        "solution": "Check the main power cable and wall outlet connection."
    },
    {
        "problem": {"won_t_turn_on": False, "strange_noise": True, "indicator_light_off": False},
        "solution": "Inspect internal cooling fan for obstructions or dust buildup."
    }
]

```

```

    },
    {
        "problem": {"won_t_turn_on": True, "strange_noise": True, "indicator_light_off": False},
        "solution": "Reset the device circuit breaker and verify fuse integrity."
    }
]

print("Existing Case Base loaded with 3 past cases.")
print("Define a new problem to test Case-Based Reasoning:")

# Interactive prompt for new problem features
p_power = input("Does the device refuse to turn on? (y/n, default: y): ").strip().lower() != 'n'
p_noise = input("Is the device making a strange noise? (y/n, default: n): ").strip().lower() == 'y'
p_light = input("Is the indicator light off? (y/n, default: y): ").strip().lower() != 'n'

new_problem = {
    "won_t_turn_on": p_power,
    "strange_noise": p_noise,
    "indicator_light_off": p_light
}

print("\nNew Problem Features:", new_problem)

# Similarity matching function
def retrieve_most_similar_case(new_prob, cases):
    best_match = None
    max_similarity = -1
    for case in cases:
        common_features = 0
        total_features = len(new_prob)
        for k, v in new_prob.items():
            if case["problem"].get(k) == v:
                common_features += 1
        similarity = common_features / total_features
        if similarity > max_similarity:
            max_similarity = similarity
            best_match = case
    return best_match, max_similarity

# Task 3C: Reuse Step
best_case, similarity_score = retrieve_most_similar_case(new_problem, case_base)

```

```

reused_solution = best_case["solution"]
print(f"\n[Reuse Step] Most similar case retrieved (Match Score: {similarity_score *
100:.0f}%):")
print(f" - Suggested Solution: {reused_solution}")

# Task 3D: Revise Step (Interactive prompt)
print("\n[Revise Step]")
user_adjustment = input("Do you want to modify this solution? (Type your custom
adjustment or press Enter to accept): ").strip()
if user_adjustment:
    final_solution = user_adjustment
    print("-> Solution revised manually by user.")
else:
    final_solution = reused_solution
    print("-> Solution kept unchanged.")

# Task 3E: Retain Step
new_case = {
    "problem": new_problem,
    "solution": final_solution
}
case_base.append(new_case)
print(f"\n[Retain Step] New case successfully added to the Case Base.")
print(f"Total cases in Case Base now: {len(case_base)}")
print("=" * 50)

```

```

... === PART 1: Knowledge Representation Setup ===
Let's set up your initial Knowledge Base (KB).
Enter current temperature (e.g., 34): 21
Enter humidity percentage (e.g., 80): 18
Is it raining? (true/false, default: true): false
Is motion detected? (true/false, default: true): true
Is the door open? (true/false, default: false): false

[Output] Knowledge Base initialized successfully:
- temperature: 21
- humidity: 18
- is_raining: False
- motion_detected: True
- door_open: False
-----

=== PART 2: Rule-Based Reasoning (RBR) ===
Rule-Based Reasoning Actions:
- No rules triggered with current facts.
-----

=== PART 3: Case-Based Reasoning (CBR) ===
Existing Case Base loaded with 3 past cases.
Define a new problem to test Case-Based Reasoning:
Does the device refuse to turn on? (y/n, default: y): n
Is the device making a strange noise? (y/n, default: n): n
Is the indicator light off? (y/n, default: y): y

New Problem Features: {'won_t_turn_on': False, 'strange_noise': False, 'indicator_light_off': True}

[Reuse Step] Most similar case retrieved (Match Score: 67%):
- Suggested Solution: Check the main power cable and wall outlet connection.

[Revise Step]
Do you want to modify this solution? (Type your custom adjustment or press Enter to accept): 14
-> Solution revised manually by user.

[Retain Step] New case successfully added to the Case Base.
Total cases in Case Base now: 4
=====

```

Comparative Reflection

Which system (RBR or CBR) was easier to implement? Why?

Rule-Based Reasoning (RBR) is generally easier to implement because it relies on straightforward conditional (if-then) statements and direct fact lookups. The logic is deterministic and simple to code using basic Python functions and dictionaries.

Which is more flexible in solving unstructured problems?

Case-Based Reasoning (CBR) is significantly more flexible for unstructured or novel problems because it does not require an exhaustive set of predefined rules. Instead, it relies on historical examples and similarity matching to adapt solutions to unfamiliar scenarios.

Which is better for situations with continuously changing data?

RBR is typically better suited for real-time environments with continuously changing operational data (such as automated IoT monitoring systems), provided the rule set is well-maintained. CBR can also adapt if new cases are continuously retained, but it requires a robust database of past experiences.

Real-World Applications:

Rule-Based Reasoning (RBR): Medical diagnostic expert systems (like MYCIN), tax calculation software, and automated router firewall security rules.

Case-Based Reasoning (CBR): Customer support ticketing systems (matching customer error logs to past resolved tickets), real estate property valuation, and legal reasoning software.